OPERATIONS GUIDE

Technical Baseline Scorecard

A controlled starting-point assessment for coding, system design, explanation quality, and the first correction to test.

Measure observable interview behavior under consistent conditions without turning one attempt into a readiness verdict.

VERSION 1.1

FORMAT GUIDE + WORKED EXAMPLES

STATUS LEARNER READY

OPERATIONS / TECHNICAL-BASELINE-SCORECARD

MEASURE THE START

A baseline captures behavior, not identity

Use the same conditions, observable dimensions, and evidence notes so the next attempt can show whether a specific correction worked.

CORE IDEA

The baseline is a starting measurement, not a readiness score. Record what happened, where the explanation broke, and the earliest decision that would improve the next attempt.

01	02	03	04
Prompt	**Conditions**	**Evidence**	**Correction**
One coding prompt and one system-design prompt.	Fixed time, tools, recording method, and no hidden assistance.	Quotes, timestamps, tests, assumptions, and missed decisions.	One behavior to front-load in the next comparable attempt.

DO	DO NOT
Score the behavior	**Avoid the label**
Clarification, decision order, invariant or requirements, correctness, trade-offs, tests, failure handling, and explanation.	Do not turn one attempt into 'good at coding,' 'bad at design,' or a fabricated percent-ready number.

Three-point evidence scale

Score	Meaning	Evidence standard
0 - Missing	The behavior did not appear when it was needed.	Name the moment and consequence.
1 - Inconsistent	It appeared late, needed prompting, or was incomplete.	Quote or timestamp the partial behavior.
2 - Reliable	It appeared unaided, at the right moment, and changed the answer.	Record the decision and resulting quality.

CODING BASELINE

Score the decisions that make a solution trustworthy

Use the neutral prompt below so the attempt is reproducible. The goal is to observe the full interview sequence, not maximize difficulty.

BASELINE PROMPT

Given a string, return the length of its longest contiguous substring with no repeated characters. Explain your state and invariant, then test empty input, all-identical characters, and a repeat that occurs after a gap.

FICTIONAL WORKED EXAMPLE - ADAPT THE METHOD, NOT THE FACTS

Dimension	Observed fictional attempt	Score + reason
Clarification	At 01:10, learner confirms substring means contiguous and returns a length; empty input returns 0.	2: contract stated unaided.
Pattern choice	At 03:20, chooses a sliding window because extending right can create a repeat that shrinking left repairs.	2: cue linked to method.
State / invariant	At 18:00, after a wrong trace, finally says the active window contains no repeated characters.	1: correct but late.
Correctness	Changes left = last[c] + 1 to max(left, last[c] + 1) after the abba case moves left backward.	1: repaired after a failing trace.
Complexity	Explains one right pass, monotonic left boundary, and a last-seen map: O(n) time and O(min(n, alphabet)) space.	2: bound tied to state.
Testing	Runs empty, bbbb, and abcabcbb; misses abba until the observer asks about an old duplicate.	1: key boundary prompted.
Communication	Approach is audible, but the pointer update is silent until the failed dry run.	1: decision checkpoint missing.

Prompt setup

30-MINUTE RUN

Recommended timing

5 minutes clarify and choose; 20 minutes implement; 5 minutes test and explain complexity.

RECORD

Evidence to save

Prompt, approach, invariant, code or notes, first missed edge, and one exact quote where reasoning became unclear.

SYSTEM-DESIGN BASELINE

Score the decisions that keep architecture explainable

Use the neutral prompt below so the baseline tests reasoning order and trade-offs rather than trivia about a particular technology.

BASELINE PROMPT

Design a notification-delivery service for product events. Support in-app and email delivery, user preferences, retries without duplicate visible sends, and delivery status. Assume 10 million monthly active users and a peak of 5,000 events per second; clarify any missing product promise before designing.

FICTIONAL WORKED EXAMPLE - ADAPT THE METHOD, NOT THE FACTS

Dimension	Observed fictional attempt	Score + reason
Requirements	At 02:00, separates in-app persistence from email delivery and asks what the user should see after a provider timeout.	2: contract clarified first.
Scale model	At 06:00, writes 5,000 peak events/s and asks about fan-out; average recipients per event is still an estimate.	2: uncertainty labeled.
Decision order	Draws queue workers at 08:00 before stating delivery semantics or replay policy.	1: component precedes reason.
Data and state	Event store owns accepted work; delivery record keyed by event, recipient, channel tracks each attempt.	2: state meanings visible.
Trade-offs	Chooses asynchronous delivery for latency isolation and accepts eventual delivery status.	2: cost stated.
Failure behavior	Retries described only at 29:00 after a crash probe; external email duplicate prevention is still uncertain.	1: incomplete boundary.
Operations	Names oldest-event age and provider-error rate, but does not define a replay check for stuck work.	1: metric without recovery proof.

Prompt setup

35-MINUTE RUN

Recommended timing

5 minutes requirements; 5 scale and API; 15 architecture/data; 10 failure, trade-offs, and close.

RECORD

Evidence to save

Requirements, estimates, architecture sketch, three assumptions, first bottleneck, one rejected alternative, and the first unclear transition.

WORKED EXAMPLE

Turn an observation into one testable correction

This fictional example shows the difference between a vague weakness label and evidence that can guide the next attempt.

Observation	Weak interpretation	Evidence-led interpretation	Next correction
Chose a hash map quickly but never stated what the values represented.	I am weak at hash maps.	Pattern choice was timely; the state invariant stayed implicit, so duplicate handling became confused.	Before coding, say: map stores value -> last index; query before insert.
Drew a queue before estimating peak load.	My architecture is shallow.	The component appeared before the load/failure requirement that justified it, making capacity and retry policy hard to defend.	State peak arrival, service rate, and retry bound before drawing the queue.
Explained the answer after finishing rather than during decisions.	I communicate badly.	Decision moments were silent; the final summary was clear but too late for an interviewer to follow the reasoning.	Use a one-sentence decision checkpoint before each major move.

CORRECTION RULE

Choose the earliest missing behavior that caused later confusion. A small front-loaded correction is easier to test than a broad promise to 'study more.'

A correction is testable when

- [ ] It can be observed at a specific moment in the next attempt.
- [ ] The prompt and rubric stay comparable.
- [ ] Success does not depend on knowing the exact same answer in advance.
- [ ] You can name what stronger evidence would look like.

WORKED BASELINE RECORD

Connect baseline scores to exact evidence

The fictional attempts below use the two prompts in this guide. Notice how the selected correction addresses a decision moment rather than labeling the learner.

FICTIONAL WORKED EXAMPLE - ADAPT THE METHOD, NOT THE FACTS

Setup	Coding attempt	System-design attempt
Prompt	Longest substring with no repeated characters; return length.	Notification delivery: in-app plus email, preferences, retries, and delivery status.
Time / tools	30 minutes; local editor; no solution lookup; observer only clarifies contract.	35 minutes; blank canvas; calculator allowed; no architecture hints.
Strongest observable behavior	03:20: explains why a window can expand and shrink while retaining one scan.	02:00: separates acceptance, persistence, and downstream delivery guarantees.
First unclear or missing decision	No explicit no-repeat invariant before code; left moves backward on abba.	Queue chosen before delivery/retry promise; crash-after-send case unresolved.
Quote / timestamp / artifact	18:00: "The active window must have no repeats." Saved abba trace and failed test.	29:00: "If acceptance succeeded but the response was lost, I need provider support or reconciliation."
Consequence in the answer	Late pointer repair compresses testing; sample success initially hides the bug.	Failure follow-up exposes a gap between at-least-once work and visible email effects.

FICTIONAL WORKED EXAMPLE - ADAPT THE METHOD, NOT THE FACTS

Selected first correction (one total)	Where it must appear	Evidence of success
Before choosing mechanics, state the invariant or user-visible delivery guarantee and one event that could break it.	After contract clarification, before the first loop or component box.	In a comparable re-test, the learner states the guarantee unaided and uses it to resolve one boundary probe without silently changing it.

FINISH THE TASK

Choose one correction and schedule the re-test

The baseline is complete when the next attempt has a defined behavior to test - not when every dimension receives a high score.

01	02	03	04
Run	**Observe**	**Select**	**Re-test**
Use fixed conditions and save the artifacts.	Score dimensions and quote the first breakdown.	Choose one early behavior with downstream impact.	Use a comparable prompt within 48-72 hours.

Do not compare totals blindly

A coding 10 and a design 10 do not mean the same thing, and a two-point movement can be noise if prompt difficulty or conditions changed. Compare dimension evidence and decision moments, not a single combined number.

Common mistakes

+ Choosing a prompt so hard that no complete interview sequence appears.
+ Changing the rubric after seeing an uncomfortable result.
+ Scoring from memory without quotes, timestamps, tests, or artifacts.
+ Selecting several corrections, which makes improvement impossible to attribute.
+ Treating a baseline as a prediction of hiring outcome.

YOUR NEXT ACTION

Write the correction as an observable sentence, choose the comparable re-test prompt now, and place it on the calendar before returning to the roadmap.

Completion check

- [ ] Two prompts were attempted under recorded conditions.
- [ ] Both scorecards contain evidence, not only numbers.
- [ ] The earliest important breakdown is named.
- [ ] One correction and one re-test condition are scheduled.

APPLY THE METHOD TO YOUR OWN EVIDENCE

After your own two attempts: copy the setup labels, preserve one quote or timestamp per score, identify the earliest missing decision, and choose one correction before scheduling a comparable re-test.